UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

PROGRAMACION III
BACK END

Anexo:

Guía práctica APIs RESTful

Guía Practica

2° Año – 3° Cuatrimestre

CC BY ND NC

Índice

Guía de buenas prácticas para el diseño y especificación de APIs RESTful 2

¿Qué es una API RESTful? 2

Principios de diseño de APIs RESTful 3

1. Recursos y naming 3
2. Verbos HTTP 3
3. Códigos de estado HTTP 4
4. Formato de respuestas 5
    Buenas prácticas: 5
5. Manejo de errores 6
    Buenas prácticas: 6
6. Versionado de la API 6
    ¿Cuándo crear una nueva versión? 6
7. Query params y path params 7
    Path Params 7
    Query Params 7
9. Documentación de la API 8
    Buenas prácticas 9
    Herramientas recomendadas 9

Guía de buenas prácticas para el diseño y especificación de APIs RESTful

Esta guía tiene como objetivo brindar un conjunto de buenas prácticas para diseñar APIs RESTful de manera clara, consistente y profesional. Está pensada para estudiantes que están dando sus primeros pasos en el desarrollo backend y necesitan comprender los principios fundamentales que guían el diseño de una buena API.

¿Qué es una API RESTful?

Una **API RESTful** (Representational State Transfer) es una interfaz que permite la comunicación entre diferentes sistemas a través del protocolo HTTP, utilizando una serie de principios como:

- Uso de **recursos** identificables por URLs
- Uso de **verbos HTTP** para describir acciones
- **Statelessness**: cada petición es independiente
- Uso de **formatos estándar** como JSON para las respuestas

Estas características permiten que las APIs sean predecibles, reutilizables y fáciles de entender tanto para quienes las desarrollan como para quienes las consumen.

Principios de diseño de APIs RESTful

1. Recursos y naming

Los recursos representan entidades del dominio del sistema, como usuarios, productos, pedidos, etc. Las rutas deben reflejar claramente la estructura de estos recursos.

Buenas prácticas:

- Usar nombres **sustantivos en plural** para los recursos: por ejemplo, `/users` para representar el conjunto de usuarios.
- Evitar el uso de verbos en la URL. La acción está dada por el verbo HTTP:
  - Correcto: `/users`, `/orders`
  - Incorrecto: `/getUser`, `/createOrder`
- Usar rutas jerárquicas para representar relaciones entre recursos:
  - `/users/42/orders`: pedidos del usuario con ID 42

2. Verbos HTTP

Los verbos HTTP indican la operación que se desea realizar sobre un recurso.

Uso recomendado:

- `GET`: obtener recursos
- `POST`: crear un nuevo recurso
- `PUT`: reemplazar un recurso existente por completo
- `PATCH`: actualizar parcialmente un recurso
- `DELETE`: eliminar un recurso

Ejemplo:

- **GET** `/users` → lista todos los usuarios
- **GET** `/users/42` → obtiene el usuario con ID 42
- **POST** `/users` → crea un nuevo usuario
- **PUT** `/users/42` → reemplaza el usuario 42
- **DELETE** `/users/` → elimina el usuario 42

3. Códigos de estado HTTP

Los códigos de estado HTTP permiten que el servidor indique al cliente el resultado de una solicitud. Comunican si la operación fue exitosa, si ocurrió un error del cliente o del servidor, o si se requiere realizar alguna acción adicional. Usarlos correctamente es fundamental para una comunicación clara y eficiente entre cliente y servidor.

Categorías de códigos de estado:

- **1xx - Informativos**: Indican que la solicitud fue recibida y el proceso continúa. No se usan comúnmente en APIs REST.
  - Ejemplo: `100 Continue`
- **2xx - Éxito**: La solicitud fue recibida, entendida y procesada correctamente.
  - `200 OK`: Solicitud exitosa, se devuelve contenido.
  - `201 Created`: Recurso creado con éxito (usualmente luego de un POST).
  - `204 No Content`: Solicitud exitosa, pero no hay contenido que devolver (común en DELETE).
- **3xx - Redirecciones**: El cliente debe realizar acciones adicionales para completar la solicitud. Tampoco son frecuentes en APIs REST.
  - `301 Moved Permanently`
  - `302 Found`
- **4xx - Errores del cliente**: El cliente hizo algo mal, como enviar datos inválidos o intentar acceder a recursos no permitidos.
  - `400 Bad Request`: La solicitud está mal formada o los datos no son válidos.
  - `401 Unauthorized`: El cliente no está autenticado.
  - `403 Forbidden`: El cliente está autenticado, pero no tiene permiso para acceder al recurso.
  - `404 Not Found`: El recurso solicitado no existe.
- **5xx - Errores del servidor**: Indican que el servidor falló al procesar una solicitud válida.
  - `500 Internal Server Error`: Error inesperado en el servidor.
  - `502 Bad Gateway, 503 Service Unavailable`: Problemas con la infraestructura del servidor.

El uso correcto de los códigos de estado estandarizados permite a los clientes automatizar respuestas, realizar manejo de errores más robusto, y depurar con mayor facilidad. Además, mejora la interoperabilidad al seguir las convenciones ampliamente aceptadas por herramientas y bibliotecas HTTP. Indican el resultado de la solicitud, proporcionando feedback estandarizado al cliente.

4. Formato de respuestas

Las respuestas de una API deben ser **claras, estructuradas, predecibles y consistentes**. Esto implica no solo utilizar un formato estándar como JSON, sino también aplicar buenas prácticas en la organización jerárquica de los datos, en la definición precisa de los tipos de datos, y en el uso uniforme del estilo de nomenclatura.

Buenas prácticas:

Usar estructuras jerárquicas claras

Las respuestas deben reflejar correctamente la relación entre entidades. Evitá respuestas planas que mezclen distintos niveles de información. Por ejemplo, si una entidad tiene datos relacionados (como un usuario con una lista de direcciones), estos deben anidarse adecuadamente.

Ejemplo correcto:

```
{
  "id": 42,
  "name": "María Gómez",
  "email": "maria@example.com",
  "addresses": [
    {
      "street": "Av. Siempre Viva 123",
      "city": "Springfield",
      "zipCode": "12345"
    }
  ]
}
```

Mantener consistencia en el estilo de nombres (*naming conventions*)

Elegí un estándar para el formato de nombres de propiedades en los objetos JSON (por ejemplo, `camelCase` o `snake_case`) y usalo de forma **consistente** en toda la API.

- `camelCase`
- `snake_case`

5. Manejo de errores

Una API bien diseñada no solo debe retornar respuestas exitosas correctamente formateadas, sino también manejar y comunicar los errores de forma **consistente, estructurada y útil para el consumidor**. Los errores deben permitir identificar fácilmente qué ocurrió, por qué ocurrió, y cómo solucionarlo si es posible.

Buenas prácticas:

- Usar códigos HTTP correctos
- Devolver errores estructurados en JSON
- Incluir un código de error y un mensaje descriptivo
- No exponer trazas de error internas ni detalles del stack

Ejemplo

```
{
  "error": "ValidationError",
  "message": "Los datos enviados no son válidos.",
  "status": 422,
  "timestamp": "2025-05-19T14:45:00Z",
  "path": "/api/users",
  "details": [
    { "field": "email", "message": "El campo email es obligatorio." },
    { "field": "password", "message": "La contraseña debe tener al menos 8 caracteres." }
  ]
}
```

6. Versionado de la API

El versionado de la API es un aspecto **crítico para garantizar su evolución sin romper la compatibilidad** con aplicaciones que ya están en producción. A medida que una API crece o cambia, es inevitable que algunas respuestas, rutas o comportamientos se modifiquen. El versionado permite introducir esos cambios sin afectar a los clientes existentes, brindando estabilidad y previsibilidad.

Recomendación: Versionar en la URL:

- `/api/v1/users`

¿Cuándo crear una nueva versión?

Debés generar una nueva versión cuando haya cambios incompatibles (breaking changes), como:

- Cambios en el formato de respuesta.
- Eliminación de campos o endpoints.
- Modificación del comportamiento esperado (por ejemplo, una ruta deja de aceptar ciertos datos).

7. Query params y path params

En una API REST, los parámetros pueden incluirse en la URL de dos formas principales: **path params** (parámetros de ruta) y **query params** (parámetros de consulta). Ambos se usan para enviar información al servidor, pero tienen propósitos distintos y deben utilizarse de manera **consistente y semánticamente correcta**.

Path Params

Se utilizan para **identificar recursos únicos** dentro de una colección. Forman parte de la jerarquía del recurso, por eso deben respetar la estructura lógica y no deben usarse para filtros o configuraciones.

Ejemplos:

`GET /users/15` # Obtener el usuario con ID 15

`GET /orders/20/ítems` # Obtener los ítems de la orden 20

Definición esperada:

`GET /users/{id}`

`GET /orders/{id}/items`

Buenas prácticas:

- Usarlos **solo cuando el recurso es único**.

El tipo de dato debe ser **coherente** (por ejemplo, si es numérico, se espera `integer` y no `string`).

Query Params

Se usan para **modificar el comportamiento de una consulta**, generalmente para **filtrar, ordenar, buscar o paginar**.

Ejemplos:

`GET /users?active=true`

`GET /products?category=books&sort=price_desc`

`GET /orders?page=2&limit=10`

8. Paginación, ordenamiento y filtros

Cuando una colección de datos puede contener cientos o miles de elementos (por ejemplo, usuarios, productos o publicaciones), devolver todos los resultados en una sola respuesta no es eficiente ni escalable. La paginación permite:

- Mejorar el rendimiento de la API.
- Reducir el uso de memoria en el cliente y servidor.
- Aumentar la velocidad de carga de las interfaces.
- Hacer que la navegación de datos sea más manejable y estructurada.

Por estas razones, la paginación es **obligatoria** en todos los endpoints que devuelvan listas extensas de recursos.

El enfoque más común y simple para manejar esto es mediante **paginación con `limit` y `offset`**.

- `limit`: indica la **cantidad máxima de elementos** que se desean obtener por página.
- `offset`: indica **cuántos elementos se deben omitir** desde el inicio de la colección.

Por ejemplo, si una colección tiene 100 elementos y hacemos la siguiente consulta:

```
GET /products?limit=10&offset=20
```

El servidor devolverá los productos desde la posición 21 hasta la 30 (recordando que el offset es cero-based).

9. Documentación de la API

Una API sin documentación clara y actualizada es difícil de entender, integrar y mantener. La documentación es la **puerta de entrada** para quienes consumen el servicio, ya sean otros equipos, clientes externos o incluso miembros del mismo equipo en el futuro.

Una buena documentación permite:

- Comprender fácilmente qué recursos ofrece la API.
- Saber cómo interactuar con esos recursos (qué métodos usar, qué datos enviar, qué esperar de respuesta).
- Detectar errores de integración más rápidamente.
- Reducir la dependencia del equipo de desarrollo para tareas repetitivas de soporte.

Recomendación:

- Documentar cada endpoint: método, path, parámetros, ejemplos de respuestas
- Usar herramientas como **OpenAPI/Swagger** para generar documentación interactiva

Buenas prácticas

Documentar todos los endpoints

Cada endpoint debe incluir:

- Método HTTP y URL.
- Descripción breve de su función.
- Parámetros requeridos y opcionales (path, query, body).
- Ejemplos de request y response en formato JSON.
- Códigos de estado HTTP esperados (`200`, `400`, `404`, etc.).

Especificar tipos y validaciones

Indicar claramente los tipos de datos (string, integer, boolean, etc.), restricciones (longitud, formato, enumeraciones) y si los campos son obligatorios u opcionales.

Proveer ejemplos claros

Los ejemplos deben ser realistas, completos y fáciles de copiar y ejecutar. Ayudan a entender el uso del endpoint sin necesidad de leer toda la descripción.

Acompañar con versionado y notas de cambios

Si la API tiene versiones o endpoints en desuso, debe indicarse. Es útil incluir notas de cambios para entender qué se modificó entre versiones.

Mantener la documentación actualizada

Debe evolucionar junto con el código. Idealmente, se mantiene en el mismo repositorio o como parte del proceso de desarrollo.

Herramientas recomendadas

- **OpenAPI (Swagger):** Permite describir la API en un formato estándar (YAML o JSON), generar documentación navegable y clientes automáticos.
- **Postman:** Permite mantener una colección de endpoints documentados y